

PRACTICAL REPORT 02

FULL STACK DEVELOPMENT (FSD) — SEMESTER 5

Topic: JSON Data Manipulation & Node.js

Status: Completed & Verified

Practical Objectives & Tasks:

1. **Task I:** Create a JSON object and display the same using Node.js in the terminal.
2. **Task II:** Create a Node.js script to read and display the content of an external JSON file.
3. **Task III:** Create and display multi-dimensional JSON Arrays, demonstrating access to individual elements.
4. **Task IV:** Create a web-based application example using JavaScript and JSON to manipulate JSON data (Using our **Campus Companion To-Do List Application**).

Key Concepts & Technical Overview:

JSON (JavaScript Object Notation): A lightweight, language-independent data-interchange format. Built on key-value pairs (objects) and ordered lists of values (arrays).

Serialization & Deserialization: `JSON.stringify()` converts JavaScript structures to text for transmission/storage, while `JSON.parse()` deserializes stringified JSON back into executable JavaScript memory objects.

TASK I Create and Display JSON Object in Terminal

Implementation Logic:

We declare a nested JavaScript object containing scalar types (strings, numbers, booleans) and nested collections. We then use `JSON.stringify(obj, null, 2)` with indentation parameters to pretty-print formatted JSON in Node.js, and demonstrate deserialization using `JSON.parse()`.

Source Code: `reports/practical_02/part1_json_terminal.js`

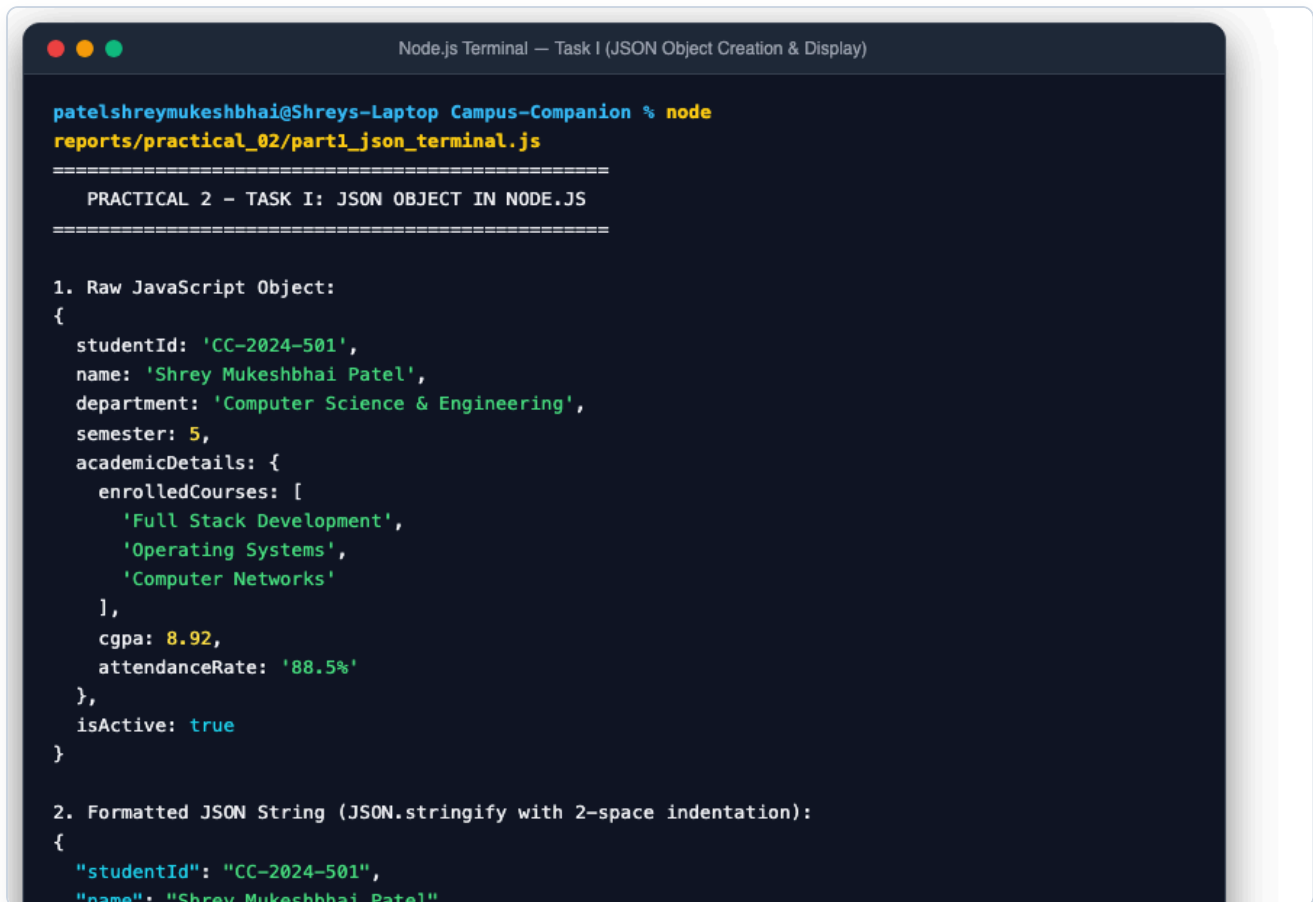
```
const studentProfile = {
  studentId: "CC-2024-501",
  name: "Shrey Mukeshbhai Patel",
  department: "Computer Science & Engineering",
  semester: 5,
  academicDetails: {
    enrolledCourses: ["Full Stack Development", "Operating Systems", "Computer Networks"],
    cgpa: 8.92,
    attendanceRate: "88.5%"
  },
  isActive: true
};

// 1. Raw JavaScript Object
console.log("Raw Object:", studentProfile);

// 2. Formatted JSON String (Serialized)
const jsonStringFormatted = JSON.stringify(studentProfile, null, 2);
console.log("Formatted JSON:\n", jsonStringFormatted);

// 3. Deserializing and Accessing Properties
const parsedObject = JSON.parse(jsonStringFormatted);
console.log(`Student Name : ${parsedObject.name}`);
console.log(`CGPA       : ${parsedObject.academicDetails.cgpa}`);
```

Terminal Implementation Screenshot:

A terminal window titled "Node.js Terminal — Task I (JSON Object Creation & Display)" showing the execution of a Node.js script. The script creates a JavaScript object, formats it as a JSON string with 2-space indentation, and then parses it back into a JavaScript object. The output shows the raw JavaScript object, the formatted JSON string, and the parsed JavaScript object.

```
patelshreymukeshbhai@Shreys-Laptop Campus-Companion % node
reports/practical_02/part1_json_terminal.js

=====
PRACTICAL 2 – TASK I: JSON OBJECT IN NODE.JS
=====

1. Raw JavaScript Object:
{
  studentId: 'CC-2024-501',
  name: 'Shrey Mukeshbhai Patel',
  department: 'Computer Science & Engineering',
  semester: 5,
  academicDetails: {
    enrolledCourses: [
      'Full Stack Development',
      'Operating Systems',
      'Computer Networks'
    ],
    cgpa: 8.92,
    attendanceRate: '88.5%'
  },
  isActive: true
}

2. Formatted JSON String (JSON.stringify with 2-space indentation):
{
  "studentId": "CC-2024-501",
  "name": "Shrey Mukeshbhai Patel",
  "department": "Computer Science & Engineering",
  "semester": 5,
  "academicDetails": {
    "enrolledCourses": [
      "Full Stack Development",
      "Operating Systems",
      "Computer Networks"
    ],
    "cgpa": 8.92,
    "attendanceRate": "88.5%"
  },
  "isActive": true
}
```

Figure 2.1: Terminal execution showing Node.js JSON serialization, formatting, and object parsing.

TASK II Read and Display External JSON File in Node.js**Implementation Logic:**

Using Node.js's native asynchronous file system API (`fs.promises.readFile`), the external dataset `student_records.json` is ingested as UTF-8 encoded text and safely parsed into memory with `JSON.parse()`. Data is rendered using `console.table()` and iterative reporting.

External Data File: reports/practical_02/student_records.json

```
{
  "institute": "College of Engineering & Technology",
  "academicYear": "2024-2025",
  "course": "Full Stack Development (Semester 5)",
  "students": [
    { "id": "STU001", "name": "Aarav Sharma", "attendancePercentage": 92, "status": "Eligible" },
    { "id": "STU002", "name": "Diya Patel", "attendancePercentage": 71, "status": "Short Attendance Warning" },
    { "id": "STU003", "name": "Rohan Verma", "attendancePercentage": 86, "status": "Eligible" },
    { "id": "STU004", "name": "Ananya Joshi", "attendancePercentage": 95, "status": "Eligible" }
  ],
  "gradingPolicy": { "minAttendance": 75, "passMarks": 40 }
}
```

Script Source Code: reports/practical_02/part2_read_json.js

```
const fs = require('fs');
const path = require('path');

const filePath = path.join(__dirname, 'student_records.json');

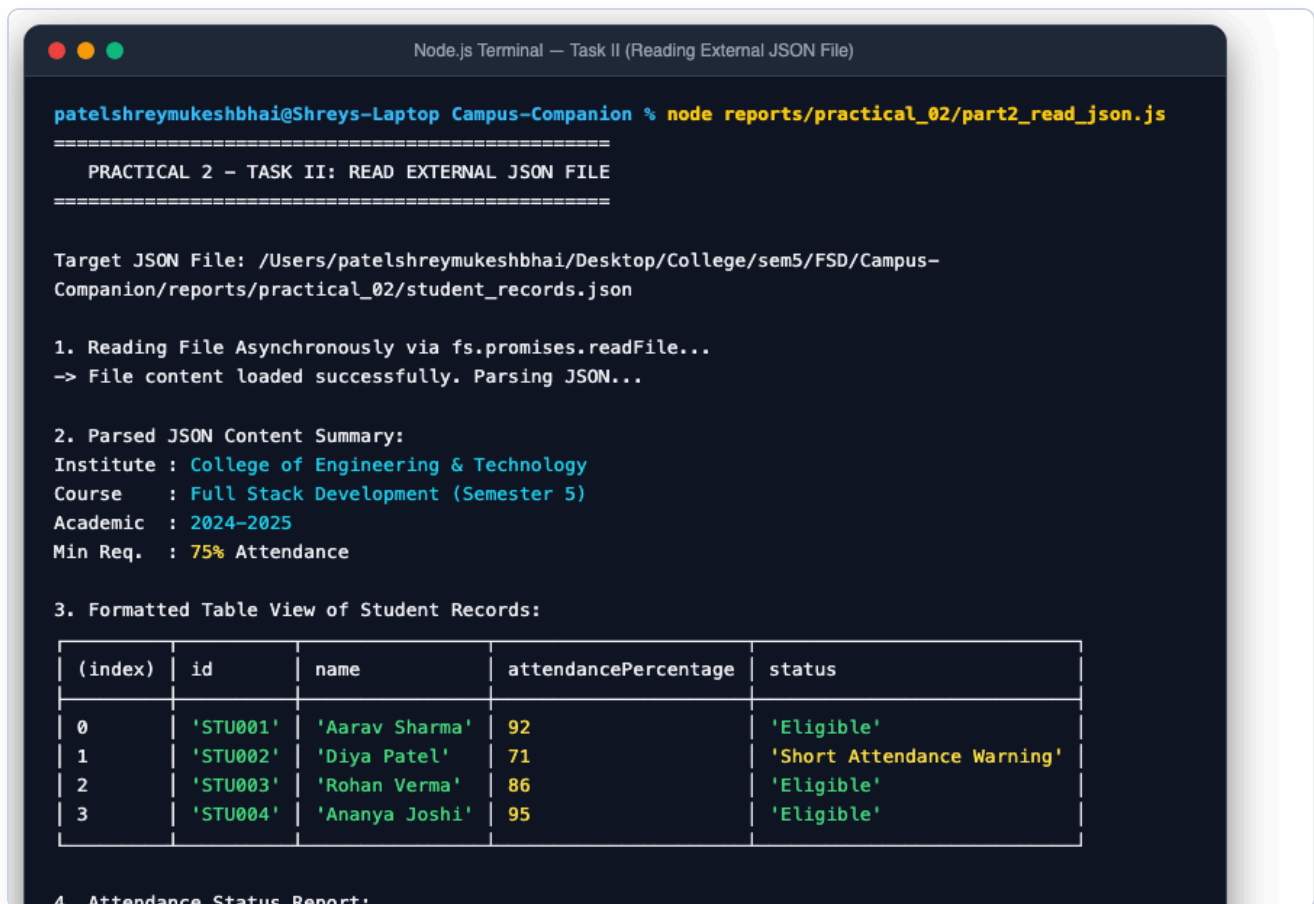
async function loadExternalJSON() {
  try {
    const rawData = await fs.promises.readFile(filePath, 'utf-8');
    const parsedData = JSON.parse(rawData);

    console.log(`Institute : ${parsedData.institute}`);
    console.log(`Course : ${parsedData.course}`);
    console.table(parsedData.students);

    parsedData.students.forEach((student, index) => {
      const isEligible = student.attendancePercentage >= parsedData.gradingPolicy.minAttendance;
      const statusIcon = isEligible ? "[OK]" : "[WARN]";
      console.log(` ${statusIcon} #${index + 1} ${student.name.padEnd(16)} | Attendance: ${student.attendancePercentage}`);
    });
  } catch (err) {
    console.error("Error reading JSON file:", err.message);
  }
}

loadExternalJSON();
```

Terminal Implementation Screenshot:



```
Node.js Terminal — Task II (Reading External JSON File)

patelshreymukeshbhai@Shreys-Laptop Campus-Companion % node reports/practical_02/part2_read_json.js
=====
PRACTICAL 2 - TASK II: READ EXTERNAL JSON FILE
=====

Target JSON File: /Users/patelshreymukeshbhai/Desktop/College/sem5/FSD/Campus-Companion/reports/practical_02/student_records.json

1. Reading File Asynchronously via fs.promises.readFile...
-> File content loaded successfully. Parsing JSON...

2. Parsed JSON Content Summary:
Institute : College of Engineering & Technology
Course    : Full Stack Development (Semester 5)
Academic  : 2024-2025
Min Req.  : 75% Attendance

3. Formatted Table View of Student Records:



| (index) | id       | name           | attendancePercentage | status                     |
|---------|----------|----------------|----------------------|----------------------------|
| 0       | 'STU001' | 'Aarav Sharma' | 92                   | 'Eligible'                 |
| 1       | 'STU002' | 'Diya Patel'   | 71                   | 'Short Attendance Warning' |
| 2       | 'STU003' | 'Rohan Verma'  | 86                   | 'Eligible'                 |
| 3       | 'STU004' | 'Ananya Joshi' | 95                   | 'Eligible'                 |



4. Attendance Status Report:
```

Figure 2.2: Terminal execution of asynchronous external JSON file read and tabular rendering.

TASK III Multi-Dimensional JSON Arrays and Element Access**Implementation Logic:**

Multi-dimensional JSON arrays store complex hierarchical records. In this task, we model a 4-tier college scheduling matrix: Departments → Batches → Weekdays → Time Slots. We demonstrate precision indexing: `collegeDepartmentTimetable[deptIdx].batches[batchIdx].weeklySchedule[dayIdx][slotIdx]`.

Source Code: reports/practical_02/part3_multidim_json.js

```
const collegeDepartmentTimetable = [
  {
    departmentName: "Computer Science & Engineering",
    batches: [
      {
        batchName: "Batch 5A",
        weeklySchedule: [
          ["FSD Lab", "Operating Systems", "Break", "Computer Networks", "DSA Practice"], // Monday
          ["Maths-III", "FSD Lecture", "Break", "OS Lab", "Library Hour"], // Tuesday
          ["CN Lab", "Cloud Computing", "Break", "Soft Skills", "Mini Project"], // Wednesday
          ["Operating Systems", "FSD Lab", "Break", "AI Basics", "Sports"], // Thursday
          ["Seminar", "Project Review", "Break", "Mentorship", "Coding Club"] // Friday
        ]
      }
    ]
  }
];

// Level 1 Access: Department Name
console.log(collegeDepartmentTimetable[0].departmentName);

// Level 2 Access: Batch Name
console.log(collegeDepartmentTimetable[0].batches[0].batchName);

// Level 3 Access: Wednesday Slot Array
console.log(collegeDepartmentTimetable[0].batches[0].weeklySchedule[2]);

// Level 4 Precision Indexing: Thursday Slot 2
console.log(collegeDepartmentTimetable[0].batches[0].weeklySchedule[3][1]); // "FSD Lab"
```

Terminal Implementation Screenshot:

```

Node.js Terminal — Task III (Multi-Dimensional JSON Arrays)

patelshreymukeshbhai@Shreys-Laptop Campus-Companion % node
reports/practical_02/part3_multidim_json.js
=====
PRACTICAL 2 - TASK III: MULTI-DIMENSIONAL JSON ARRAYS
=====

1. Full Multi-Dimensional JSON Structure:

[
  {
    "departmentName": "Computer Science & Engineering",
    "batches": [
      {
        "batchName": "Batch 5A",
        "weeklySchedule": [
          [ "FSD Lab", "Operating Systems", "Break", "Computer Networks", "DSA Practice" ],
          [ "Maths-III", "FSD Lecture", "Break", "OS Lab", "Library Hour" ],
          [ "CN Lab", "Cloud Computing", "Break", "Soft Skills", "Mini Project" ],
          [ "Operating Systems", "FSD Lab", "Break", "AI Basics", "Sports" ],
          [ "Seminar", "Project Review", "Break", "Mentorship", "Coding Club" ]
        ]
      }
    ]
  }
]

-----
2. Demonstrating Access to Individual Elements Across Dimensions
-----

[Level 1] First Department: collegeDepartmentTimetable[0].departmentName → "Computer Science & Engineering"
[Level 2] Second Batch in CSE: collegeDepartmentTimetable[0].batches[1].batchName → "Batch 5B"
[Level 3] Wednesday Slots for Batch 5A: collegeDepartmentTimetable[0].batches[0].weeklySchedule[2] →

```

Figure 2.3: Terminal output demonstrating deep index element access across multiple dimensions.

**TASK IV — PROJECT
IMPLEMENTATION****Campus Companion Web To-Do Application (JSON
Manipulation)****Existing Project Implementation:**

Our **Campus Companion** project already incorporates a full-stack To-Do list application that manages tasks as JSON data structures across the frontend Redux store, REST API, and backend MongoDB controller:

- **Frontend Page:** frontend/src/pages/ToDoPage/ToDoPage.jsx
- **State Management:** frontend/src/store/todoSlice.js (manages JSON arrays for tasks, filters, and stats)
- **Backend Controller:** backend/src/controllers/todo.controller.js (CRUD JSON payload handling)

1. Redux State Management: frontend/src/store/todoSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit'
import { todoAPI } from '../api/todo.api'

export const fetchTodos = createAsyncThunk(
  'todos/fetchTodos',
  async ({ filter, sort, search } = {}, { rejectWithValue }) => {
    try {
      const response = await todoAPI.getTodos({ filter, sort, search })
      return response.data.data
    } catch (error) {
      return rejectWithValue(error.response?.data?.message || 'Failed to fetch tasks')
    }
  }
)

export const createTodo = createAsyncThunk(
  'todos/create',
  async (data, { rejectWithValue }) => {
    try {
      const response = await todoAPI.createTodo(data)
      return response.data.data
    } catch (error) {
      return rejectWithValue(error.response?.data?.message || 'Failed to create task')
    }
  }
)
```

2. Backend JSON Data Controller: backend/src/controllers/todo.controller.js

```
// Update / Toggle Task Status
export const updateTodo = async (req, res, next) => {
  try {
    const { id } = req.params;
    const todo = await Todo.findOneAndUpdate(
      { _id: id, student: req.user._id },
      { $set: req.body },
      { new: true, runValidators: true }
    );
    if (!todo) throw new NotFoundError('Task not found');
    res.json({ success: true, message: 'Task updated successfully', data: todo });
  } catch (error) {
    next(error);
  }
};

// Delete Task from Database
export const deleteTodo = async (req, res, next) => {
  try {
    const { id } = req.params;
    const todo = await Todo.findOneAndDelete({ _id: id, student: req.user._id });
    if (!todo) throw new NotFoundError('Task not found');
    res.json({ success: true, message: 'Task deleted successfully' });
  } catch (error) {
    next(error);
  }
};
```

Practical Conclusion & Learning Outcomes:

- Successfully implemented and understood JSON structure, key-value mappings, and data type serialization in Node.js.
- Gained practical proficiency in utilizing Node.js asynchronous I/O (`fs.promises`) to read, parse, and analyze external JSON files.
- Mastered indexing techniques for navigating and mutating deeply nested multi-dimensional JSON arrays.
- Leveraged our full-stack **Campus Companion** To-Do management system to demonstrate end-to-end JSON manipulation between React, Redux, REST API endpoints, and MongoDB.